

Expressions Reference

Expressions add dynamic logic to templates — formatting values, calculating totals, applying conditions, and controlling visibility. This page is the complete function reference.

Expressions are built on [Jexl \(JavaScript Expression Language\)](#). Standard Jexl expressions are fully compatible with Doctavian. Use the [Jexl Playground](#) to test expression logic quickly — note that Doctavian's custom functions listed below won't be available in the playground.

Syntax

Expressions follow this notation in templates:

```
{!$Expression}
```

For example, to display the current date in short format:

```
{!$format(now(), 'date', 'short')}
```

Value wrapping

Every field value inside an expression is treated as a string by default, regardless of its source type. Wrap values in the appropriate conversion function before performing type-specific operations. Numeric parameters in string functions accept both integer and string values — for example, `charAt(field, 1)` and `charAt(field, '1')` return the same result.

```
{!$toDecimal(Invoice[0].TotalAmount) + 10}      → numeric addition, not string concatenation
{!$Invoice[0].TotalAmount + 10}                  → "1250.510" (string concatenation)
{!$format(date(Invoice[0].DueDate), 'date', 'medium')}
{!$charAt(Invoice[0].Reference, 1)}               → same as charAt(field, '1')
```

Array functions

Function	Returns	Description
<code>count(Object)</code>	integer	Number of records in the collection
<code>isEmpty(Object)</code>	boolean	true if the collection has no records
<code>isNotEmpty(Object)</code>	boolean	true if the collection has at least one record
<code>max(Object, "field")</code>	any	Largest value of the specified field across all records
<code>min(Object, "field")</code>	any	Smallest value of the specified field across all records
<code>sum(Object, "field")</code>	number	Sum of the specified field across all records
<code>join(Object, "field", "separator")</code>	string	Joins field values from all records into a delimited string
<code>groupBy(Object, "field", "alias")</code>	object	Groups records by the specified field, assigning each group to the given alias

Examples (assuming 3 line items with *TotalPrice* values of 400, 350, 500 and recipients *Jane Smith, John Doe*)

```
{!$count(Invoice[0].LineItems)}      → 3
{!$sum(Invoice[0].LineItems, "TotalPrice")} → 1250
{!$join(Invoice[0].Recipients, "Name", ", ")} → Jane Smith, John Doe
```

String functions

Function	Returns	Description
<code>isBlank(field)</code>	boolean	true if the field is null or empty
<code>contains(field, 'value')</code>	boolean	true if the field contains the specified value — case-sensitive
<code>concat('a', 'b', field)</code>	string	Concatenates all provided arguments — accepts strings, fields, and expressions
<code>toLowerCase(field)</code>	string	Returns the value in lower case
<code>toUpperCase(field)</code>	string	Returns the value in upper case
<code>trim(field)</code>	string	Removes leading and trailing whitespace

Function	Returns	Description
<code>replace(field, 'find', 'replace')</code>	string	Replaces the first occurrence of a pattern — the replacement can be a field value
<code>replaceAll(field, 'find', 'replace')</code>	string	Replaces all occurrences of a pattern — can be nested for multiple replacements
<code>substring(field, start, end)</code>	string	Extracts a substring between two indexes — end is optional
<code>slice(field, start, end)</code>	string	Extracts a portion of the string
<code>split(field, 'separator', limit)</code>	array	Splits the string into an ordered list of substrings — limit is optional
<code>startsWith(field, 'value', position)</code>	boolean	true if the field starts with the given value — position is optional
<code>endsWith(field, 'value', position)</code>	boolean	true if the field ends with the given value — position is optional
<code>indexOf(field, 'value', position)</code>	integer	Position of the first occurrence — position is optional
<code>lastIndexOf(field, 'value', position)</code>	integer	Position of the last occurrence — returns -1 if not found at or before position
<code>charAt(field, index)</code>	string	Character at the specified index
<code>charCodeAt(field, index)</code>	integer	UTF-16 code unit of the character at the specified index — returns an integer between 0 and 65535
<code>repeat(field, count)</code>	string	Repeats the string value the specified number of times — accepts a field or a literal

Examples (assuming *BuyerFirstName* = "Jane", *BuyerLastName* = "Smith", *Status* = "overdue", *InvoiceNumber* = "INV-0042", *DiscountCode* = "")

```
{!$concat(Invoice[0].BuyerFirstName, ' ', Invoice[0].BuyerLastName)} → Jane Smith
{!$isBlank(Invoice[0].DiscountCode)} → true
{!$toUpperCase(Invoice[0].Status)} → OVERDUE
{!$replace(Invoice[0].InvoiceNumber, 'INV-', '#')} → #0042
{!$replaceAll(replaceAll(replaceAll(field, '.', '#'), ',', '.'), '#', ',')} → swaps decimal and thousand separators
```

Number functions

Function	Returns	Description
<code>toDecimal(field)</code>	decimal	Converts a value to a decimal for arithmetic operations
<code>toPercent(value, decimals)</code>	string	Formats a decimal as a percentage — wrap field values in <code>toDecimal()</code> first, e.g. <code>toPercent(toDecimal(field), 2)</code>
<code>round(value, 'round', decimals)</code>	number	Rounds to the specified number of decimal places
<code>setScale(value, decimals)</code>	string	Displays a number with the exact number of decimal places — <i>decimals</i> accepts an integer or string
<code>compact(value, 'short' 'long', decimals)</code>	string	Formats a large number in compact notation — e.g. 58735298 → 58.735M ('short') or 58.735 million ('long')
<code>index()</code>	integer	Current iteration index inside a Repeater or Table — in a Repeater returns 0-based index; in XLSX returns the actual row number
<code>row()</code>	integer	Current row number — XLSX and Google Sheets only
<code>column()</code>	string	Current column letter — XLSX and Google Sheets only

Examples (assuming *TotalAmount* = "1250.5", *TaxRate* = "0.123", *Amount* = "99.567")

```
{!$toDecimal(Invoice[0].TotalAmount) * 1.2} → 1500.6
{!$toPercent(toDecimal(Invoice[0].TaxRate), 2)} → 12.30%
{!$toPercent(0.123, 2)} → 12.30%
{!$round(toDecimal(Invoice[0].Amount), 'round', 2)} → 99.57
{!$setScale(toDecimal(Invoice[0].Amount), 4)} → 99.5670
{!$compact(58735298, 'short', 3)} → 58.735M
{!$compact(34500, 'long', 5)} → 34.50000K
{!$index()} → 0 (first iteration)
```

Date functions

Function	Returns	Description
<code>now()</code>	datetime	Current date and time — wrap in <code>format()</code> to display
<code>date(field)</code>	date	Casts a date string to a date value — required before passing to <code>format()</code>

Function	Returns	Description
dateTime(field)	datetime	Casts a datetime string — required before passing to format()
time(field)	time	Extracts the time portion of a datetime field — wrap in format() to display
addDays(date, n)	date	Adds n days — accepts now(), a field directly, or a date()-wrapped field
addMonths(date, n)	date	Adds n months — same input options as addDays
addYears(date, n)	date	Adds n years — same input options as addDays

date() and dateTime() are required when passing a field to format(). However, addDays(), addMonths(), and addYears() accept a raw field value directly without wrapping.

date() and dateTime() parsing is locale-dependent — the same string can resolve to different dates depending on the active locale. Always verify the parsed result matches what you expect before relying on it in production.

Examples (assuming DueDate = "2025-08-24", IssueDate = "2025-07-25", CreatedDate = "2025-08-24T14:30:00")

!\$format(now(), 'date', 'medium')}	→ Aug 24, 2025
!\$format(date(Invoice[0].DueDate), 'date', 'short')}	→ 8/24/2025
!\$format(addDays(now(), 10), 'date', 'medium')}	→ Sep 3, 2025
!\$addDays(Invoice[0].CreatedDate, 1)	→ raw datetime + 1 day
!\$addDays(now(), 1) != now()	→ true

Format function

The format() function controls the display format of dates, datetimes, times, and numbers.

Date and datetime formats

Expression	Output
format(date, 'date', 'short')	8/24/2025
format(date, 'date', 'medium')	Aug 24, 2025
format(date, 'date', 'long')	August 24, 2025
format(now(), 'time', 'short')	12:10 AM
format(dateTime, 'dateTime', 'short')	8/24/2025, 2:30 PM
format(dateTime, 'dateTime', 'medium')	Aug 24, 2025, 2:30:00 PM
format(dateTime, 'dateTime', 'long')	August 24, 2025 at 2:30:00 PM GMT+2
format(dateTime, 'date', 'short')	8/24/2025
format(date, 'raw', 'yyyy-MM-dd')	2025-08-24
format(dateTime, 'raw', 'dd-MM-yyyy/HH:mm:ss')	24-08-2025/14:30:00

The output of 'date', 'long' may match 'date', 'medium' depending on the active locale. For example, with locale en-GB both return 18 May 2020. Use 'raw' for a guaranteed specific format.

The 'raw' format type accepts any custom pattern using standard date/time format tokens — useful when you need a specific structure not covered by the predefined short/medium/long options.

Watch out for year format tokens. yyyy is the calendar year of the date, while YYYY is the calendar year of the week — these differ for dates at the very start or end of a year. For example, 30 December 2024 falls in the first week of 2025, so yyyy returns 2024 but YYYY returns 2025. Use yyyy unless you specifically need ISO week-year behaviour.

Number formats

Expression	Output
format(toDecimal(field), 'number', '#,###')	1,250
format(toDecimal(field), 'number', '#,###.00')	1,250.00
format(toDecimal(field), 'number', '0.00')	1250.00

Comparison

Comparison operators work on numeric values — wrap field values in toDecimal() before comparing. You can also compare against the result of array functions like sum(), min(), max(), and count(), as well as date expressions.

Google Docs and Google Slides templates must use HTML entities for comparison operators — angle brackets are interpreted as HTML tags and will be stripped. Use `>` for `>`, `<` for `<`, `≥` for `>=`, and `≤` for `<=`. This restriction does not apply to DOCX, XLSX, or PPTX templates.

Function	Returns	Description	Google Docs notation
<code>toDecimal(field) < value</code>	boolean	Less than	<code>&lt;</code>
<code>toDecimal(field) > value</code>	boolean	Greater than	<code>&gt;</code>
<code>toDecimal(field) <= value</code>	boolean	Less than or equal to	<code>&le;</code>
<code>toDecimal(field) >= value</code>	boolean	Greater than or equal to	<code>&ge;</code>
<code>field == value</code>	boolean	Equal to — works on strings, numbers, and expressions	no change
<code>field != value</code>	boolean	Not equal to — works with date expressions too	no change
<code>expr1 && expr2</code>	boolean	Logical AND — both expressions must be true	no change
<code>expr1 expr2</code>	boolean	Logical OR — at least one expression must be true	no change
<code>condition ? valueIfTrue : valueIfFalse</code>	any	Ternary — returns one of two values based on a condition	no change

Examples (assuming `Amount = "150"`)

```

{!$toDecimal(Invoice[0].Amount) < 100}           → false
{!$toDecimal(Invoice[0].Amount) > 100}           → true
{!$toDecimal(Invoice[0].Amount) == 150}          → true
{!$toDecimal(Invoice[0].Amount) != 100}          → true
{!$toDecimal(Invoice[0].Amount) > count(Invoice[0].LineItems)} → true
{!$toDecimal(Invoice[0].Amount) < sum(Invoice[0].LineItems, 'TotalPrice')} → false
{!$addDays(now(), 1) != now()}                   → true
{!$1 < 3 && 4 == 2}                               → false
{!$1 < 3 || 4 == 2}                               → true
{!$(Invoice[0].Status == "Overdue") ? "Payment required" : "All good"} → All good
{!$Invoice[0].Status == "Overdue" || Invoice[0].Amount >= 5000 ? "Review" : "OK"} → OK

```

Comparison expressions are commonly used as the hidden parameter on elements to conditionally show or hide content:

```

<mdoc:paragraph name="discount" hidden="{!$toDecimal(Invoice[0].Total) < 10000}">
  ...
</mdoc:paragraph>

```

Logical

Function	Returns	Description
<code>toBoolean(expression)</code>	boolean	Evaluates the expression and returns true or false as a string value — useful when a boolean result needs to be rendered as text in the document

Example (assuming `Amount = "150"`)

```

{!$toBoolean(toDecimal(Invoice[0].Amount) < 100)} → false

```

Native Jexl expressions

Standard Jexl operators and expressions work in Doctavian without any wrapping function. This includes the `in` operator for substring checks, the ternary operator for conditional values, and combined logical expressions.

Examples

```

{!$" " ? "Full" : "Empty"}           → Empty
{!$"Opp" in "Opportunity" ? "Yes" : "No"} → Yes
{!$Invoice[0].Status == "Paid" ? "Thank you" : "Payment required"} → Payment required
{!$CreatedDate==now() || Amount>=5000000 || Name=='Edge SLA' ? 'Found' : 'Not found'} → Not found

```

Nested expressions

Functions can be nested — the output of one function becomes the input of another. There is no limit to nesting depth.

Examples (assuming *IssueDate* = "2025-07-25", *Quantity* = "2", *UnitPrice* = "30000")

<code>{!\$format(addDays(now(), 10), 'dateTime', 'medium')}</code>	→ Sep 3, 2025, 12:00:00 AM
<code>{!\$setScale(toDecimal(Invoice[0].LineItems[0].Quantity), '4')}</code>	→ 2.0000
<code>{!\$round(toDecimal(Invoice[0].LineItems[1].UnitPrice) / 7, 'round', 3)}</code>	→ 4,285.714
<code>{!\$replaceAll(replaceAll(replaceAll(field, '.', '#'), ',', '.'), '#', ',')}</code>	→ swaps decimal and thousand separators

Expressions with operators

Standard arithmetic operators (+, -, *, /) can be used directly inside expressions — including within function arguments and with && inside function parameters.

Examples

<code>{!\$sum(Invoice[0].LineItems, 'Quantity') + 4}</code>	→ total quantity plus 4
<code>{!\$addDays(now(), 1-1)}</code>	→ today's date
<code>{!\$addDays(now(), 8/2)}</code>	→ adds 4 days
<code>{!\$addDays(now(), 3*1)}</code>	→ adds 3 days
<code>{!\$Invoice[0].CustomerName + ', ' + Invoice[0].City}</code>	→ Acme Corp, London
<code>{!\$contains(Invoice[0].Name, 'E' && 'S')}</code>	→ true

Expression with alias

When a Data Source query defines a custom object alias — for example, mapping *SalesRep* as *SalesAgent* — expressions can reference that alias directly in the template. This allows you to use meaningful, domain-specific names rather than the raw object or field names from your data source.

Example (assuming *SalesAgent* is an alias defined in the query)

`{!$"Rom" in SalesAgent[0].Name ? "Correct" : "Wrong User"}` → Wrong User

The alias is defined in the Data Source query using the Query Builder's field aliasing or Expression Fields. See [Data Sources](#) for how to configure aliases in your query.